REINFORCEMENT LEARNING

TD Prediction, On-Policy and Off-Policy Algorithms

Subject	Reinforcement Learning Laboratory / Assignment
Topics	Temporal-Difference Prediction, SARSA, Q-Learning
Environment	FrozenLake and a brief Taxi Problem example
Prepared for	Academic submission

Assignment overview

This assignment explains how Temporal-Difference (TD) learning estimates state values and how TD control methods learn action values. It focuses on the two important control approaches: SARSA, which is on-policy, and Q-Learning, which is off-policy. The discussion follows the concepts and environments used in the supplied RL laboratory notebooks.

1. Introduction to Reinforcement Learning

Reinforcement Learning (RL) is a machine-learning approach in which an agent learns how to act by interacting with an environment. At each time step, the agent observes a state, selects an action, receives a reward, and moves to a new state. The objective is to learn behaviour that maximizes the total long-term reward.

Main elements of RL

- **Agent:** The learner or decision-maker.
- **Environment:** The world in which the agent operates, such as FrozenLake or Taxi.
- **State:** The current situation of the environment.
- **Action:** A choice available to the agent.
- **Reward:** Feedback that indicates how useful an action or outcome was.
- **Policy:** The strategy used by the agent to select actions.

2. Temporal-Difference (TD) Prediction

TD Prediction estimates the value of states while following a policy. Unlike Monte Carlo methods, TD learning does not need to wait until an episode ends. It updates a value estimate after each transition using the reward received and the estimate of the next state. This is called bootstrapping.

TD(0) update equation

$$V(S\blacksquare) \leftarrow V(S\blacksquare) + \alpha\,[R\blacksquare\blacksquare\blacksquare + \gamma V(S\blacksquare\blacksquare\blacksquare) - V(S\blacksquare)]$$

Here, V(S■) is the current estimate of the value of state S■; α is the learning rate; R■■■ is the reward observed after the action; γ is the discount factor; and V(S■■■) is the estimated value of the next state. The expression inside the brackets is the TD error.

Procedure for TD Prediction

- Initialize the value of each state, usually to zero.
- Start an episode and observe the current state.
- Choose an action according to the policy being evaluated.
- Perform the action and observe the reward and next state.
- Update the current state's value using the TD(0) equation.
- Move to the next state and repeat until the episode terminates.
- Repeat for many episodes so that the value estimates improve.

Example: FrozenLake

In the supplied laboratory notebook, FrozenLake is used as the environment. The environment has discrete states arranged as a grid. A random policy selects actions, and a dictionary stores the estimated value for each state. After each transition, the TD update adjusts the value estimate. The resulting table shows the learned values for the states under the selected policy.

3. On-Policy Control: SARSA

SARSA stands for State–Action–Reward–State–Action. It is an on-policy TD control algorithm because it learns the value of the same policy that is used to choose actions during interaction. SARSA updates a state-action value using the next action actually selected by the current policy.

SARSA update equation

$$Q(S■,A■) \leftarrow Q(S■,A■) + \alpha\,[R■■■ + \gamma Q(S■■■,A■■■) - Q(S■,A■)]$$

The important feature is Q(S■■■,A■■■): the next action A■■■ is selected using the current behaviour policy, such as an ε-greedy policy. Therefore, the update accounts for the actions the agent may actually take while learning.

Procedure for SARSA

- Initialize Q(S,A) for every state-action pair.
- Start an episode and select an action using an ε-greedy policy.
- Execute the action and observe the reward and next state.
- Select the next action from the next state using the same ε-greedy policy.
- Update Q(S,A) using the SARSA equation.
- Set the current state and action to the next state and next action.
- Continue until the episode ends, then repeat for multiple episodes.

Exploration and exploitation

An ε-greedy policy explores by choosing a random action with probability ε and exploits by choosing the action with the highest current Q-value otherwise. Exploration helps the agent discover alternatives; exploitation uses what it has learned so far.

4. Off-Policy Control: Q-Learning

Q-Learning is an off-policy TD control algorithm. It learns the value of a greedy target policy while the agent may use a different behaviour policy, such as ε-greedy exploration. Its update uses the maximum Q-value available in the next state rather than the value of the next action actually selected.

Q-Learning update equation

$$Q(S■,A■) \leftarrow Q(S■,A■) + \alpha\,[R■■■ + \gamma\,\max■\,Q(S■■■,a) - Q(S■,A■)]$$

The term max■ Q(S■■■,a) represents the best estimated action value in the next state. The update therefore moves toward the greedy target, even when the behaviour policy selected an exploratory action.

Procedure for Q-Learning

- Initialize the Q-table with values such as zero.
- Observe the current state and choose an action using the behaviour policy.
- Execute the action and observe the reward and next state.
- Find the maximum Q-value among all actions available in the next state.
- Update the current state-action value using the Q-Learning equation.
- Move to the next state and repeat until the episode terminates.
- Repeat over many episodes to improve the Q-table.

5. Comparison of TD Prediction, SARSA and Q-Learning

Feature	TD Prediction	SARSA	Q-Learning
Main purpose	Estimate state values	Learn action values	Learn action values
Type	Prediction	On-policy control	Off-policy control
Update target	Reward + next-state value	Reward + value of next chosen action	Reward + maximum next-state action value
Policy relationship	Evaluates a specified policy	Behaviour and target policy are the same	Behaviour and target policy can differ
Stored values	V(S)	Q(S,A)	Q(S,A)

6. Application: Taxi Problem Using Q-Learning

The Taxi environment provides a practical example of off-policy learning. A taxi must pick up a passenger at one designated location and drop the passenger at another. In the supplied notebook, a successful drop-off earns +20 reward, each time step costs −1, and an illegal pickup or drop-off costs −10. The goal is to learn a route and sequence of actions that completes the task efficiently while avoiding illegal actions.

The Q-table stores a value for each state-action pair. At every step, an ε-greedy policy selects an action, the environment returns the next state and reward, and Q-Learning updates the value using the maximum action value of the next state. Training over many episodes allows the agent to improve its decisions.

7. Advantages and Limitations

Advantages

- TD methods learn from each transition without waiting for the end of an episode.
- SARSA and Q-Learning can learn action-selection strategies from experience.
- Q-tables are straightforward to understand in small, discrete environments.
- ε-greedy action selection provides a simple balance between exploration and exploitation.

Limitations

- Tabular methods become large when the number of states or actions increases.
- Learning results depend on parameters such as learning rate, discount factor and exploration rate.
- Training may require many episodes, and early behaviour can be inefficient.
- The simple tabular approach is not directly suitable for very large or continuous state spaces.

8. Conclusion

Temporal-Difference learning updates value estimates using immediate rewards and estimates of future value. TD Prediction evaluates a policy by estimating state values. SARSA is an on-policy control method because it uses the next action selected by its current policy. Q-Learning is an off-policy method because it updates toward the best next-state action value, independently of the next action actually taken. FrozenLake illustrates TD Prediction and control, while the Taxi problem demonstrates how Q-Learning can learn a practical sequence of actions from rewards and penalties.

References

- Reinforcement Learning laboratory notebook supplied with the assignment: TD Prediction, TD Control (SARSA) and Q-Learning using FrozenLake.
- Reinforcement Learning laboratory notebook supplied with the assignment: Solving the Taxi Problem using Q-Learning.
- Sutton, R. S. and Barto, A. G., Reinforcement Learning: An Introduction, 2nd edition, MIT Press.